

rank the most valuable statically analyzed relationships the semantic engine could uncover for open forge codebases

I'd rank them by one criterion: "does this relationship materially change what a contributor should investigate or do next?" That puts the most valuable relationships above merely interesting repository statistics. Existing repository-mining work supports the usefulness of change coupling, ownership, hotspots, and developer/code relationships, but Oculus can push these much further by joining them to forge activity and investigations.

1. **Event** → **affected semantic entities**. For any commit, PR, issue-linked change, or failing test: exactly which modules, types, functions, APIs, tests, configuration elements, and dependency boundaries are implicated. This is the fundamental relationship because almost every later investigation starts here.
2. **Semantic entity** → **historical evolution**. For a function/type/module/API, reconstruct its meaningful lineage: when introduced, major modifications, refactors, reversions, ownership changes, and behavioral shifts. This lets a contributor ask not merely "where is this?" but "why does this exist in its current form?"
3. **Issue/PR** → **code and behavioral surface**. Establish likely trace links between forge artifacts and concrete implementation entities using explicit references, historical fixes, changed paths, symbol overlap, tests, and textual evidence. Traceability between issue artifacts and source history is already a major research area; Oculus could make it interactive rather than merely analytical.
4. **Code** → **change coupling**. Which things repeatedly change together even though the dependency/import graph doesn't say they are related? This is one of the highest-value discoveries because it exposes implicit architecture: hidden interfaces, duplicated responsibilities, synchronization requirements, and places where a seemingly local fix probably isn't local. Change-coupling analysis is a proven repository-mining technique, and modern tools are already combining it with static dependency graphs.
5. **Change** → **probable propagation surface**. Given a proposed or historical change to X, what other symbols, tests, modules, APIs, generated artifacts, documentation, or configurations are likely to require corresponding changes? This is the relationship I'd make central to patch generation: "if you touch this, these are the things you should investigate."
6. **Issue** → **related issues/PRs** → **common underlying entity**. Not merely textual duplicates, but reports that converge on the same symbols, tests, commits, behavioral paths, or architectural boundary. This can reveal that five apparently separate complaints are actually one missing abstraction.
7. **Current change** → **historical precedent**. "This PR is doing something similar to these four previous changes." More importantly: "three succeeded, one was reverted, and the reverted one failed because of X." This is potentially one of Oculus's killer relationships because it gives an agent and contributor access to the project's institutional memory.
8. **Entity** → **tests** → **behavior**. Which tests exercise this code, what behavior those tests establish, and which historical changes modified the test alongside the implementation.

This creates a bridge from source structure to observable behavior and makes experiments much more intelligent.

9. **Contributor** → **semantic area** → **historical expertise**. Not just “who has the most commits,” but who has repeatedly modified, reviewed, discussed, or fixed this particular subsystem. That can surface appropriate people to consult, likely reviewers, and the project's actual knowledge distribution. Research has shown that repository history can support useful developer/issue expertise recommendations.
10. **Code** → **ownership and knowledge concentration**. Which important or volatile areas depend heavily on one contributor's historical knowledge? This is especially interesting for an open project because it tells a contributor where documentation, tests, or deliberate knowledge transfer may be needed.
11. **Entity** → **architectural boundary crossings**. Identify functions/modules/packages that repeatedly reach across otherwise distinct subsystems. Combined with historical coupling, this can reveal “soft architecture” that isn't represented in imports or types: the places where the project is actually crossing boundaries.
12. **Issue/feature** → **project trajectory**. Does a proposed contribution align with what the codebase has actually been moving toward? The engine can establish the evidence: recent API direction, refactors, deprecated paths, changing dependencies, repeated feature requests, and accepted/rejected approaches. The agent can then interpret that evidence as “this seems aligned with the current direction.”
13. **Patch** → **precedent and contradiction**. For a candidate patch, find prior code that supports the approach and prior decisions that contradict it. This is enormously useful for agents: instead of asking “can I implement this?”, they can ask “does this implementation agree with the project's accumulated design evidence?”
14. **Subsystem** → **instability**. Identify areas where churn, reversions, bug fixes, issue activity, test failures, and architectural changes concentrate. A contributor seeing an apparently simple issue might discover that they're actually entering the project's most unstable subsystem.
15. **Feature request** → **existing latent capability**. Find requests that appear to require new functionality but where much of the necessary machinery already exists. This could be particularly valuable for contributors: “the requested feature is 70% present; these two existing abstractions and this dormant code path suggest a smaller implementation.”
16. **Code duplication** → **synchronized evolution**. Don't merely find duplicate code; find duplicates that repeatedly change together or receive parallel fixes. Those are much stronger candidates for abstraction because the repository's history demonstrates that they are behaviorally coupled. This combination of static structure and historical behavior is exactly the kind of fusion I'd want Oculus's engine to specialize in.
17. **Documentation/community statement** → **implementation reality**. Find discrepancies between what the project says a feature/API does and what the source actually does. Then connect those discrepancies to issues and recent changes. This turns documentation problems into investigations rather than lint warnings.
18. **Activity event** → **unusual situation**. Detect events that deserve attention even when nobody explicitly asks: an unusually large architectural change, a PR modifying a historically sensitive area, a contributor changing code outside their usual area, a

formerly stable subsystem suddenly experiencing churn, or an issue appearing immediately after a relevant commit.

The top five are the foundation I'd build first: **event-to-code mapping, semantic history, forge-to-code traceability, change coupling, and propagation surface**. Together they let Oculus answer the most important contributor question: "Given this thing that just happened, what else in the project should I understand before I act?"

The really powerful relationships are actually combinations of these primitives. For example: **issue → affected function → recent commit → historically coupled function → related test → previous PR → contributor**. Every edge can be deterministic and individually inspectable, while the *meaning of the path* can be supplied by the agent. That is where I think the architecture becomes substantially more interesting than existing repository analytics: tools such as CodeLore already demonstrate how far deterministic history/static-analysis fusion can go; Oculus can make those relationships the starting point for an interactive investigation rather than the final output.

And I would make **relationship provenance mandatory**. If Oculus tells me "these issues are related," I should be able to expand it and see: shared symbol, 0.81 lexical similarity, same historical fixing PR, overlapping test, and temporal proximity. Then the agent can say "I think these are manifestations of the same architectural problem," but it cannot disguise that interpretation as a deterministic fact. That separation between **evidence graph** and **interpretation graph** is, in my view, one of the most important design decisions for the entire semantic engine.

perhaps give me a simple list of the facts i need to start pulling from activity streams and repository stores of open projects, ordered from most fundamental to most complex, to align my workload for developing these features for the plugin

I would build the data foundation in roughly this order, where each level enables the next:

1. Repository identity: forge, owner, repository, URL, default branch, forks/mirrors, visibility, description.
2. Git references: branches, tags, HEADs, releases, commit IDs, ancestry, timestamps, authors and committers.
3. Source files: paths, language, size, generated/vendor status, hashes, current revision.
4. Source symbols: modules, namespaces, types, functions, methods, constants, macros, classes, interfaces, etc., with definitions and stable identities.

5. Symbol relationships: definitions, references, calls, imports, implementations, inheritance, containment, dependencies.
6. Commits → changed files and symbols: additions, deletions, modifications, renames, line ranges, semantic entities affected.
7. Tests → code: test files, test cases, symbols exercised, packages/targets, pass/fail results where available.
8. Build/dependency structure: packages, crates/modules, external dependencies, build targets, generated artifacts, configuration relationships.
9. Forge activity objects: issues, PRs, discussions, comments, reviews, labels, milestones, reactions, releases.
10. Forge cross-references: issue ↔ PR, issue ↔ commit, PR ↔ commit, PR ↔ issue, discussion ↔ issue/PR, explicit closing/fixing references.
11. Forge event history: opened, closed, reopened, labeled, assigned, reviewed, commented, merged, reverted, released, etc., with timestamps.
12. Forge participants: users associated with issues, PRs, reviews, commits, discussions, and releases.
13. Code ownership history: who has historically modified, reviewed, or maintained particular files, symbols, modules, and subsystems.
14. Change history per semantic entity: introduction, modifications, refactors, renames, deletions, reintroductions, reversions, and periods of activity/inactivity.
15. Co-change relationships: files/symbols that repeatedly change together despite lacking explicit dependency relationships.
16. Structural impact relationships: callers, callees, dependents, affected tests, dependency paths, API consumers, and likely propagation surfaces.
17. Issue/PR ↔ code evidence: textual symbol/path references, historical fixing commits, changed-code overlap, test overlap, temporal relationships, and structural proximity.
18. Issue/PR ↔ issue/PR similarity: shared symbols, files, terminology, historical fixes, affected behavior, discussion overlap, and common architectural areas.
19. Behavioral relationships: tests, failures, diagnostics, regressions, commits, and source entities connected into reproducible chains.
20. Architectural relationships: subsystem boundaries, dependency crossings, high-coupling areas, shared abstractions, duplicated responsibilities, and recurring cross-boundary changes.
21. Temporal dynamics: churn, instability, repeated fixes, reversions, abandoned approaches, rapidly changing APIs, emerging areas, and dormant areas becoming active.
22. Community/code relationships: contributors associated with particular subsystems, reviewers with particular areas, recurring collaborators, concentrated knowledge, and community attention around technical areas.
23. Historical precedent: current changes/issues mapped to similar previous changes, successful approaches, rejected approaches, reverted approaches, and their outcomes.
24. Repository anomalies: unusual churn, suspicious regressions, unexplained test failures, documentation/code disagreement, orphaned issues, stale PRs, duplicated requests, and changes inconsistent with recent project patterns.

25. Derived investigations: evidence bundles connecting an initiating event to relevant facts, relationships, hypotheses, unanswered questions, experiments, candidate patches, test results, and eventual outcomes.
26. Agent hypotheses: explicitly separate from facts, with the evidence that motivated each hypothesis, confidence, requested verification, subsequent supporting/contradicting observations, and eventual resolution.

The first really important milestone is around **1–17**. At that point you have enough substrate to make an issue or PR genuinely intelligent: “this event touches these symbols, these symbols have this history, these tests exercise them, these people have worked on them, these commits and forge objects are connected to them, and these are the likely propagation surfaces.”

Then **18–24** is where Oculus starts becoming qualitatively different from a GitHub/Codeberg client. You're no longer just retrieving project information; you're detecting patterns in the project's development and community dynamics.

Finally, **25–26** is where the agent layer becomes truly powerful: the agent isn't being asked to discover the repository from scratch. It receives a structured situation assembled from the first 24 layers, forms hypotheses, requests additional evidence, conducts experiments, and eventually produces work that remains connected to the evidence that motivated it.